



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251189
Nama Lengkap	Alicia Luna Santoso
Minggu ke / Materi	09/ Pengolahan String dan Regular Expression

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

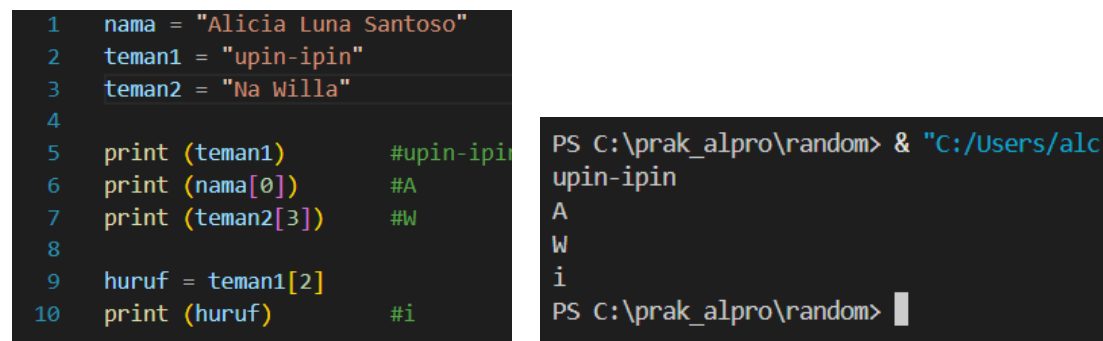
Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Link Github : https://github.com/alc357/71251189_alicia.git

PENGANTAR STRING

String adalah gabungan karakter dalam komputer untuk menyimpan kalimat. String adalah jenis data yang dapat menyimpan huruf/karakter dan disimpan dalam ASCII. String pada dasarnya adalah Kumpulan tipe data karakter/array of character/list of character.

PENGAKSESAN STRING DAN MANIPULASI STRING

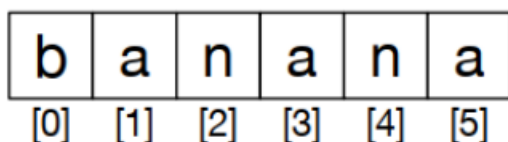


```
1  nama = "Alicia Luna Santoso"
2  teman1 = "upin-ipin"
3  teman2 = "Na Willa"
4
5  print (teman1)           #upin-ipin
6  print (nama[0])          #A
7  print (teman2[3])        #W
8
9  huruf = teman1[2]
10 print (huruf)            #i
```

```
PS C:\prak_alpro\random> & "C:/Users/alc
upin-ipin
A
W
i
PS C:\prak_alpro\random>
```

Gambar 1.1 : Pengaksesan string dan manipulasi string

String pertama dibuat dengan deklarasi variabel lalu diisi dengan data, string dapat diakses sebagai satu kesatuan dengan menyebut nama variabelnya, atau per huruf dengan menyebutkan indeksinya. **Indeks pada string dimulai dari 0** seperti layaknya pada list. Indeks merupakan bilangan bulat, bukan pecahan.



Gambar 1.2 : Bentuk string di dalam memory

OPERATOR DAN METODE STRING :

OPERATOR in

String kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil dari operator ini adalah True/False.

```
1 kalimat = "saya mau makan pecel lele"
2 data = "lele"
3 print(data in kalimat)
4 print("ayam" in kalimat )
```

```
PS C:\prak_alpro\random> & "C:/Users/alc 357/App
hon.exe" c:/prak_alpro/random/soal.py
True
False
```

Gambar 1.3 : Operator in

Pada string juga dapat dilakukan perbandingan (comparison) yang menghasilkan True dan False

```
1 if "saya" > "dia":
2     print("Ya")      #Ya
3 else:
4     print("Tidak")
5 if "dua" == "dua":
6     print("Sama")    #Sama
```

```
PS C:\prak_alpro\random> & "C:/Users/
hon.exe" c:/prak_alpro/random/soal.py
Ya
Sama
```

Gambar 1.4 : Perbandingan pada string

Fungsi len

Cara mengetahui berapa panjang (berapa jumlah karakter) dari sebuah string adalah dengan menggunakan operator len () untuk menampilkan huruf terakhir dari sebuah string kita menggunakan indeks string yang ke-len(<string>-1), sebab indeks dimulai dari 0. Contoh programnya yaitu :

```
1 kalimat = "universitas kristen duta wacana yogyakarta"
2 print(len(kalimat))
3 terakhir = kalimat[len(kalimat)-1]
4 print(terakhir) #output 'a'
5
6 #bisa juga menggunakan indeks -1
7 terakhir_versi2 = kalimat[-1]
8 print(terakhir_versi2) #output 'a'
9 #atau menggunakan indeks -2 untuk huruf terakhir kedua
10 terakhir2 = kalimat[-2]
11 print(terakhir2) #output 't'
12
```

```
PS C:\prak_alpro\random>
python.exe" c:/prak_alpr
42
a
a
t
```

Gambar 1.5 : Fungsi len

Traversing string

Menampilkan string dengan cara ditampilkan huruf demi huruf adalah dengan menggunakan loop dengan cara : dilakukan dengan akses terhadap indeks dan dilakukan tanpa akses terhadap indeks secara otomatis

```
1 kalimat = "Halo Semua"
2 i = 0
3 while i < len(kalimat):
4     print(kalimat[i],end='')
5     i += 1
6
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

PS C:\prak_alpro\random> & "C:/Users/313/python.exe" c:/prak_alpro/random
Halo Semua

Gambar 1.6 : Traversing string akses indeks

```
1 kalimat = "indonesia jaya"
2 for kal in kalimat:
3     print(kal,end='')
4
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

PS C:\prak_alpro\random> & "C:/Users/313/python.exe" c:/prak_alpro/random
indonesia jaya

Gambar 1.7 : Traversing string tanpa akses indeks.

String slice

String slice adalah menampilkan substring pada sebuah string dengan menggunakan indeks dari awal tertentu sampai akhir-1 tertentu. Sintaksnya menggunakan <string>[awal:akhir]. Bagian awal atau akhir boleh dikosongkan. Bagian awal dimulai dari 0.

```
1 kalimat = "aku bingung"
2 awal = 0
3 akhir = 6
4 print(kalimat[awal:akhir])
5 print(kalimat[7:len(kalimat)])
6 print(kalimat[:5])
7 print(kalimat[5:])
8 print(kalimat[:])
```

```
PS C:\prak_alpro\random> & "C:/Users/313/python.exe" c:/prak_alpro/random  
aku bi  
gung  
aku b  
ingung  
aku bingung  
PS C:\prak_alpro\random>
```

Gambar 1.8 : penggunaan string slice

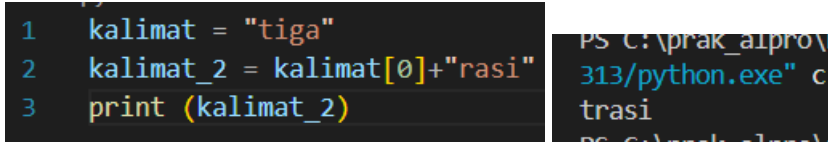
Kalimat bersifat immutable. Immutable berarti data tidak dapat diubah saat program berjalan, hanya bisa diinisialisasi saja.

```
1 kalimat = "satu"
2 kalimat[0] = "batu"
```

```
TypeError: 'str' object does not support item assignment  
PS C:\prak_alpro\random>
```

Gambar 1.9 : bukti dari immutable

Agar dapat diubah, maka dapat disimpan dalam variabel yang berbeda



```
1 kalimat = "tiga"
2 kalimat_2 = kalimat[0]+"rasi"
3 print (kalimat_2)
```

PS C:\prak_alpro\313> python.exe c:\prak_alpro\313\prak_alpro_1.py

Gambar 1.10 : String diubah dan disimpan dalam variabel yang berbeda.

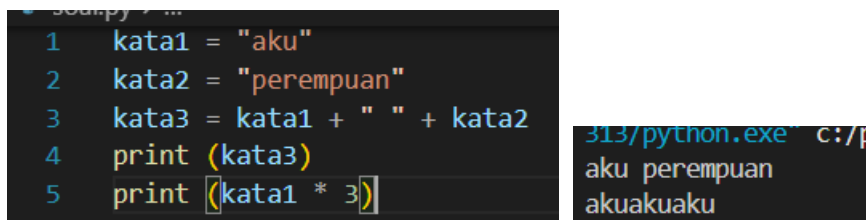
Metode yang sering digunakan adalah :

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang diinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang diinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Tabel 1.1 : method string yang sering digunakan

Operator * dan + pada String

Operator + digunakan untuk menggabungkan dua string, sedangkan operator * digunakan untuk menampilkan string sebanyak berapa kali.



```
1 kata1 = "aku"
2 kata2 = "perempuan"
3 kata3 = kata1 + " " + kata2
4 print (kata3)
5 print (kata1 * 3)
```

313/python.exe c:/p

aku perempuan
akuakuaku

Gambar 1.11 : Penggunaan operator * dan +

PARSING STRING

Parsing string adalah cara menelusuri string bagian demi bagian untuk mendapat / mengubah

bagian yang diinginkan. Contohnya mengubah format tanggal pada suatu kalimat dari menggunakan tanda hubung (-) menjadi (/) :

```
kalimat = "hari ini tanggal 15-04-2026"
hasil = kalimat.split(" ")
for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
        print(hasil2[0]+"/"+hasil2[1]+"/"+hasil2[2])
```

313/python.exe
15/04/2026

Gambar 1.12 : Mengubah format penulisan tanggal

PENGANTAR REGEX

Regular Expression adalah ekspresi pola yang berbentuk kumpulan karakter yang digunakan untuk menemukan, mencari, mengganti, dan menghapus string dengan pola tertentu. Python merupakan salah satu Bahasa yang mendukung library regex dengan cara import re. Salah satu fungsi yang mudah digunakan adalah search()

```
1 import re
2 handle=open('mbox-short.txt')
3 count = 0
4 for line in handle:
5     line=line.rstrip()
6     if re.search('From:', line):
7         count += 1
8         print(line)
9         print("Count: ",count)
```

Gambar 1.13 : Menggunakan import regex pada file mbox-short.txt

Re.search dapat digantikan dengan perintah find() pada string biasa. Pola pada contoh belum sepenuhnya menggunakan kemampuan regex

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlw@iupui.edu
From: zqian@umich.edu
From: rjlw@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
From: zqian@umich.edu
From: antranig@caret.cam.ac.uk
From: gopal.ramasammycook@gmail.com
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: louis@media.berkeley.edu
From: ray@media.berkeley.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
Count: 27
```

Gambar 1.14 : Output yang dihasilkan dari code di gambar 1.13

Kode ini juga dapat menggunakan fungsi string.startswith("From :").

META CHARACTER, ESCAPED CHARACTER, SET OF CHARACTER, DAN FUNGSI REGEX PADA LIBRARY PYTHON

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{ }	Karakter dengan arti khusus dan escaped character	\{ }d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
\$	Dakhiri dengan	this\$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{ }d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{ }d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{ }d+	Minimal 1 s/d tak terhingga karakter
{ }	Tepat sebanyak yang ada para { }	\{ }d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
	atau	\{ }d \{ }s	1 digit atau 1 spasi

Tabel 1.2 : Special character Python

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\b" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Tabel 1.3 : Escaped Character pada regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Tabel 1.4 : Himpunan Karakter pada Regex

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Tabel 1.5 : Fungsi regex pada python

Sumber : Modul 9 String dan Regex ([Modul 9 - String dan Regex.pdf](#))

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/alc357/71251189_alicia.git

SOAL 1

Source Code :

```
k1 = input("Masukan kata pertama : ").lower()
k2 = input("Masukan kata kedua : ").lower()

if len(k1) == len(k2) :
    k1_urut = sorted(k1)
    k2_urut = sorted(k2)
    if k1_urut == k2_urut :
        print(f"{k1} dan {k2} adalah anagram")
    else :
        print(f"{k1} dan {k2} bukan anagram")
else :
    print(f"{k1} dan {k2} bukan anagram")
```

Langkah – langkah :

1. Pertama, kita buat dua variabel yang berfungsi untuk menampung dua kata yang akan dibandingkan apakah anagram atau bukan. Jangan lupa untuk membuat semua nya menjadi huruf kecil menggunakan .lower()
2. Kemudian, kita buat percabangan dimana jika jumlah huruf antara kata pertama dengan kata kedua sama maka kita akan melakukan proses perbandingan. Namun jika jumlah huruf tidak sama sudah pasti 2 kata tersebut bukan anagram

3. Jika jumlah huruf kedua kata sama maka kita urutkan dulu kata 1 dan kata dua dengan perintah `sorted` yang ditampung dalam variabel, seharusnya jika kedua kata adalah anagram maka seharusnya jika diurutkan (berdasarkan a-z) kata 1 dan kata 2 sama.
4. Kemudian buat percabangan lagi yang berisi jika kata1 dan kata2 yang sudah diurutkan nilainya sama maka beri output bahwa kata 1 dan kata 2 adalah anagram namun jika tidak maka bukan anagram.

Output :

```
C:\Praktikum-Alpro---71251189\m
Masukan kata pertama : mata
Masukan kata kedua : atma
mata dan atma adalah anagram
PS C:\prak_alpro\71251189_alicia
cia/Praktikum-Alpro---71251189/m
Masukan kata pertama : ayam
Masukan kata kedua : ayah
ayam dan ayah bukan anagram
PS C:\prak_alpro\71251189_alicia
cia/Praktikum-Alpro---71251189/m
Masukan kata pertama : garam
Masukan kata kedua : madu
garam dan madu bukan anagram
```

Gambar 2.1 : Output program soal nomor 1

```
1 k1 = input("Masukan kata pertama : ").lower()
2 k2 = input("Masukan kata kedua : ").lower()
3
4 if len(k1) == len(k2) :
5     k1_urut = sorted(k1)
6     k2_urut = sorted(k2)
7     if k1_urut == k2_urut :
8         print(f"{k1} dan {k2} adalah anagram")
9     else :
10        print(f"{k1} dan {k2} bukan anagram")
11 else :
12    print(f"{k1} dan {k2} bukan anagram")
```

Gambar 2.2 : Kode Program soal nomor 1

SOAL 2

Source Code :

```
kalimat = input("Masukan kalimat-kalimatmu : ").lower()

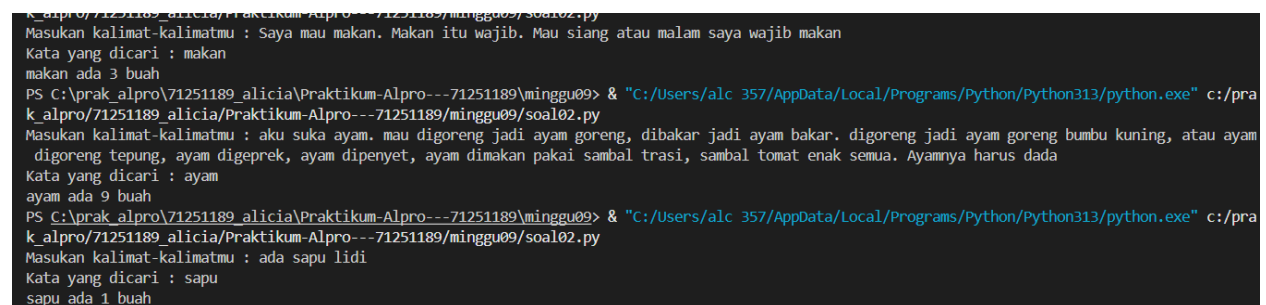
kata = input("Kata yang dicari : ").lower()
```

```
print(f"{kata} ada {kalimat.count(kata)} buah")
```

Langkah – langkah :

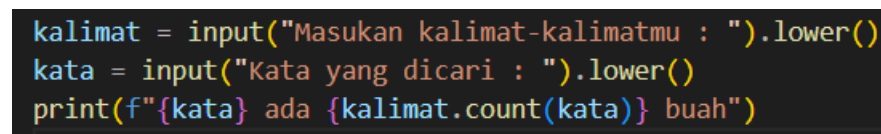
1. Pertama, kita akan meminta input dari user berupa kumpulan kata atau kalimat yang akan dijadikan objek pencarian dan ditampung dalam sebuah variabel bernama kalimat. Yang kemudian dibuat huruf kecil semua untuk mengantisipasi jika kata yang dicari berada di awal kalimat.
2. Kemudian minta user untuk menginput kata yang ingin dicari dan ditampung dalam variabel bernama kata. Yang kemudian dibuat huruf kecil.
3. Selanjutnya, kita print hasil dari perhitungan jumlah kata yang dicari dari kalimat yang diinput user dengan perintah .count()

Output :



```
k_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09\soal02.py
Masukan kalimat-kalimatmu : Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
Kata yang dicari : makan
makan ada 3 buah
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09> & "C:/Users/alc_357/AppData/Local/Programs/Python/Python313/python.exe" c:/pra
k_alpro/71251189_alicia/Praktikum-Alpro---71251189/minggu09/soal02.py
Masukan kalimat-kalimatmu : aku suka ayam. mau digoreng jadi ayam goreng, dibakar jadi ayam bakar. digoreng jadi ayam goreng bumbu kuning, atau ayam
digoreng tepung, ayam digeprek, ayam dipenyet, ayam dimakan pakai sambal trasi, sambal tomat enak semua. Ayamnya harus dada
Kata yang dicari : ayam
ayam ada 9 buah
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09> & "C:/Users/alc_357/AppData/Local/Programs/Python/Python313/python.exe" c:/pra
k_alpro/71251189_alicia/Praktikum-Alpro---71251189/minggu09/soal02.py
Masukan kalimat-kalimatmu : ada sapu lidi
Kata yang dicari : sapu
sapu ada 1 buah
```

Gambar 2.3 : Output dari soal nomor 2



```
kalimat = input("Masukan kalimat-kalimatmu : ").lower()
kata = input("Kata yang dicari : ").lower()
print(f"{kata} ada {kalimat.count(kata)} buah")
```

Gambar 2.4 : Kode soal nomor 2

SOAL 3

Source code :

```
kalimat = input("Masukan kalimat : ")

tdk_spasi = kalimat.split()

perbaikan = ' '.join(tdk_spasi)
```

```
print(f'sebelum diperbaiki : "{kalimat}"')  
print(f'Setelah diperbaiki : "{perbaikan}"')
```

Langkah – Langkah :

1. Pertama, kita akan meminta input dari user berupa kalimat yang mengandung spasi berlebihan di awal, Tengah, atau akhir kalimat dan ditampung dalam variabel kalimat
2. Kemudian, kita pecah kalimat menjadi potongan kata menggunakan fungsi `.split()`. Metode ini akan secara otomatis menghilangkan spasi dan memisahkan kata berdasarkan spasi lalu hasil ditampung dalam variabel bernama `tdk_spasi`
3. Selanjutnya, kita akan menggabungkan kembali kata yang sudah dipisah menjadi satu kalimat utuh dengan dipisahkan oleh spasi Tunggal menggunakan `.join()`. Hasil ini ditampung dalam variabel bernama `perbaikan`.
4. Terakhir, kita akan mencetak kalimat sebelum perbaikan dan hasil kalimat yang sudah diperbaiki diapit dengan tanda `""` supaya ketika spasi berlebih berada di akhir maka hasil perbaikan terlihat.

Output :

```
Masukan kalimat : Aku mau makan  
sebelum diperbaiki : "Aku mau makan  "  
Setelah diperbaiki : "Aku mau makan"  
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\  
e" c:/prak_alpro/71251189_alicia/Praktikum-Alpro---71251189/  
Masukan kalimat : aku  suka lele goreng  
sebelum diperbaiki : "aku  suka lele goreng"  
Setelah diperbaiki : "aku suka lele goreng"  
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\  
e" c:/prak_alpro/71251189_alicia/Praktikum-Alpro---71251189/  
Masukan kalimat :  halo aku alicia  
sebelum diperbaiki : "  halo aku alicia  "  
Setelah diperbaiki : "halo aku alicia"
```

Gambar 2.5 : put soal nomor 3

SOAL 4

Source code :

```

kalimat = input("Masukan kalimat : ")

kalimat_list = kalimat.split()

terpendek = kalimat_list[0]

terpanjang = kalimat_list [0]

for kata in kalimat_list :

    if len(kata) < len(terpendek) :

        terpendek = kata

    if len(kata) > len(terpanjang) :

        terpanjang = kata

print (f"terpendek : {terpendek}, terpanjang : {terpanjang}")

```

Langkah – Langkah :

1. Pertama, kita akan meminta input dari user berupa sebuah kalimat yang akan kita cari kata terpendek dan kata terpanjangnya yang ditampung dalam sebuah variabel bernama kalimat
2. Selanjutnya, kita akan memecah kalimat tersebut menjadi potongan kata menggunakan .split() dan menyimpannya dalam sebuah list bernama kalimat_list.
3. Selanjutnya kita menginisialisasi variabel terpendek dan terpanjang dengan nilai awal berupa kata pertama dari list sebagai asumsi awal perbandingan
4. Kemudian, kita akan melakukan perulangan untuk memeriksa setiap kata yang ada dalam list menggunakan for
5. Di dalam for, kita akan membandingkan panjang kata saat ini dengan panjang kata dalam variabel terpendek . Jika kata saat ini lebih pendek dari kata yang berada dalam variabel terpendek maka kita ganti nilai variabel terpendek dengan kata tersebut.

6. Kemudian kita juga membandingkan panjang kata saat ini dengan panjang kata pada variabel terpanjang. Jika kata saat ini lebih panjang, kita akan mengganti nilai variabel terpanjang dengan kata tersebut.
7. Terakhir, kita cetak hasil kata terpendek dan hasil kata terpanjang.

Output :

```
Masukan kalimat : red snakes and a black frog in the pool
terpendek : a, terpanjang : snakes
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu
7/AppData/Local/Programs/Python/Python313/python.exe" c:/prak_alp
tikum-Alpro---71251189/minggu09/soal04.py
Masukan kalimat : aku memakan tempe goreng yang sudah dimarinasi
terpendek : aku, terpanjang : dimarinasi
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu
```

Gambar 2.7 : output soal nomor 4

```
kalimat = input("Masukan kalimat : ")

kalimat_list = kalimat.split()
terpendek = kalimat_list[0]
terpanjang = kalimat_list [0]
for kata in kalimat_list :
    if len(kata) < len(terpendek) :
        terpendek = kata
    if len(kata) > len(terpanjang) :
        terpanjang = kata
print (f"terpendek : {terpendek}, terpanjang : {terpanjang}")
```

Gambar 2.8 : Kode Pemrograman soal nomor 4

SOAL 4

Source code :

```
import re

from datetime import datetime
```

```

teks = input("Masukan kalimat yang mengandung tanggal dengan format YYYY-MM-DD
: ")

dftr_tgl = re.findall(r'\d{4}-\d{2}-\d{2}', teks)

tgl_skr = datetime.now().date()

if len (dftr_tgl) == 0 :

    print("Tidak ada tanggal yang sesuai format")

else :

    for tgl_str in dftr_tgl :

        tgl_obj = datetime.strptime(tgl_str, '%Y-%m-%d')

        selisih = (tgl_skr - tgl_obj.date()).days

        print(f"{tgl_str} 00:00:00 selisih {selisih} hari")

```

Langkah – Langkah :

1. Pertama, import dua library yang diperlukan yaitu re untuk mencari pola tanggal menggunakan regular expression dan datetime untuk memanipulasi data tanggal dan menghitung selisih hari.
2. Kemudian kita meminta input user berupa kalimat yang mengandung tanggal dengan format YYYY-MM-DD dan ditampung dalam variabel bernama teks
3. Selanjutnya, cari semua pola tanggal yang sesuai dengan format menggunakan re.findall(). Pola(r' \d{4}-\d{2}-\d{2}', teks. Artinya 4 digit angka, diikuti tanda strip, 2 digit angka, diikuti tanda strip, dan 2 digit angka. Hasil pencarian akan diimpmkan dalam sebuah list bernama dftr_tgl
4. Setelah itu, kita akan mendapat tanggal hari ini (saat program dijalankan) menggunakan datetime.now().date()
5. Selanjutnya perikda apakah ada tanggal yang ditemukan atau tidak dengan menggunakan if-else. Jika panjang list dftr_tgl adalah 0 maka tidak ditemukan tanggal dan keluarkan output

6. Jika ada tanggal, maka kita proses perulangan for untuk mengakses satu per satu tanggal yang tersimpan dalam list `dftr_tgl`.
7. Setiap tanggal kita ubah dari string menjadi objek datetime dengan `datetime.strptime()` dengan format `'%Y-%m-%d'`. Kemudian simpan dalam variabel `tgl_obj`.
8. Kemudian, kita hitung selisih hari antara tanggal sekarang dengan tanggal saat ini menggunakan operasi pengurangan. Kemudian mengambil nilai harinya dengan `attribute.days` dan disimpan dalam variabel `selisih`

Output :

```
Masukan kalimat : red snakes and a black frog in the pool
terpendek : a, terpanjang : snakes
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09\soal04.py
7/AppData/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/Praktikum-Alpro---71251189\minggu09/soal04.py
Masukan kalimat : aku memakan tempe goreng yang sudah dimarinasi
terpendek : aku, terpanjang : dimarinasi
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09\soal04.py
```

Gambar 2.9 : output soal nomor 5

```
kalimat = input("Masukan kalimat : ")

kalimat_list = kalimat.split()
terpendek = kalimat_list[0]
terpanjang = kalimat_list[0]
for kata in kalimat_list:
    if len(kata) < len(terpendek):
        terpendek = kata
    if len(kata) > len(terpanjang):
        terpanjang = kata
print(f"terpendek : {terpendek}, terpanjang : {terpanjang}")
```

Gambar 2.10 : Kode Pemrograman soal nomor 5

SOAL 6

Source code :

```
import re
```

```

import random

print("masukan teks yang mengandung email (ketik - jika sudah) : ")

semua = []

while True :

    baris = input()

    if baris == "-" :

        break

    semua.append(baris)

teks = "\n".join(semua)

daftar_email      =      re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', teks)

karakter = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789"

if len(daftar_email) == 0 :

    print("tidak ditemukan email")

else :

    for email in daftar_email :

        username = email.split('@')[0]

        password = ''

        for i in range (8) :

            password += random.choice(karakter)

        print (f"{email} username : {username} , password : {password}")

```

Langkah – Langkah :

1. Pertama, kita import dua library yang diperlukan yaitu re untuk mencari pola email dan random untuk menghasilkan password acak.

2. Kemudian, kita akan menampilkan instruksi kepada user untuk memasukan teks yang mengandung email. User dapat memasukan beberapa baris teks, dan untuk mengakhiri input, user cukup menghentikan tanda strip (-)
3. Selanjutnya, kita akan menyimpan sebuah list kosong bernama semua untuk menampung setiap baris teks yang diinput.
4. Setelah itu, kita buat perulangan while true untuk meminta input dari user sampai user mengetikkan tanda strip (-)
5. Dalam perulangan, kita memeriksa apakah baris yang diinput user adalah strip, jika ya maka perulangan akan dihentikan dengan break
6. Jika baris bukan strip, maka baris akan ditambahkan ke dalam list semua menggunakan .append().
7. Setelah perulangan selesai, kita gabungkan semua baris dengan .join() dengan pemisah newline kemudian disimpan dalam variabel bernama teks
8. Selanjutnya, cari semua pola email dalam teks menggunakan re.findall(). Kita mencari kombinasi karakter huruf, angka, titik, underscore, plus, minus sebelum tanda @ kemudian tanda @ kemudian domain yaitu huruf, angka, titik, strip, lalu titik, dan diakhiri dengan ekstensi domain minimal 2 huruf, hasilnya akan disimpan dalam list bernama daftar_email
9. Langkah berikutnya kita akan menyiapkan Kumpulan karakter yang bisa digunakan untuk membuat password random. Karakter tersebut terdiri dari huruf besar A-Z, huruf kecil (a-z), dan angka (0-9) yang disimpan dalam variabel bernama karakter.
10. Kemudian, kita memeriksa email ditemukan atau tidak dengan percabangan if-else. Jika panjang list daftar_email = 0 maka kita menampilkan pesan bahwa tidak ditemukan email
11. Jika email ditemukan maka kita akan memproses email dengan perulangan for untuk mengakses semua email dalam list daftar_email

12. Untuk setiap email kita akan mengambil username yaitu bagian sebelum tanda @ dengan menggunakan `.split('@')` sehingga email akan terpecah menjadi 2 bagian kemudian kita akan mengambil indeks ke 0 saja sebagai username
13. Selanjutnya kita akan menyiapkan variabel bernama password sebagai string kosong yang nantinya akan diisi dengan 8 karakter acak.
14. Kemudian kita buat perulangan for sebanyak 8 kali untuk memilih 8 karakter secara acak dari Kumpulan karakter yang sudah disiapkan
15. Dalam perulangan, kita akan mengambil satu karakter menggunakan `random.choice(karakter)` lalu ditambahkan dalam variabel bernama password
16. Terakhir, kita cetak hasil berupa email, username, dan password yang sudah dibuat.

Output :

```
minggu09/soal06.py
masukan teks yang mengandung email (ketik - jika sudah) :
Berikut adalah daftar email dan nama pengguna dari mailing list:
anton@mail.com dimiliki oleh antonius
budi@gmail.co.id dimiliki oleh budi anwari
slamet@getnada.com dimiliki oleh slamet slumut
matahari@tokopedia.com dimiliki oleh toko matahari
-
anton@mail.com username : anton , password: 5SnVojJJ
budi@gmail.co.id username : budi , password: 7Zxr3bJY
slamet@getnada.com username : slamet , password: 8ho4bRIId
matahari@tokopedia.com username : matahari , password: zpgPIand
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu09>
```

Gambar 2.11 : output soal nomor 6

```

1  import re
2  import random
3  print("masukan teks yang mengandung email (ketik - jika sudah) : ")
4  semua = []
5  while True :
6      baris = input()
7      if baris == "-" :
8          break
9      semua.append(baris)
10     teks = "\n".join(semua)
11
12     daftar_email = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', teks)
13     karakter = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789"
14     if len(daftar_email) == 0 :
15         print("tidak ditemukan email")
16     else :
17         for email in daftar_email :
18             username = email.split('@')[0]
19             password = ''
20             for i in range(8) :
21                 password += random.choice(karakter)
22             print(f"{email} username : {username} , password: {password}")

```

Gambar 2.12 : Kode Pemrograman soal nomor 6